



ARQUITETURA DE SISTEMAS

AULA 4 - CARACTERÍSTICAS ARQUITETURAIS



REQUISITOS VS. CARACTERÍSTICAS ARQUITETURAIS

- ❑ Requisitos de Negócio (Funcionais): Funcionalidades específicas que o software deve fornecer para atender às necessidades do domínio do problema (ex.: cadastro de usuário, processamento de pedido).
- ❑ Características Arquiteturais (Qualidades): Aspectos gerais de qualidade e operação do sistema além das funcionalidades de domínio. Representam o que o sistema deve atender em termos de desempenho, segurança, escalabilidade, etc., e não o que ele faz em termos de regras de negócios.
- ❑ Responsabilidade do Arquiteto: Identificar e endereçar esses requisitos de arquitetura desde o início do projeto. Eles não aparecem diretamente nas histórias de usuário, mas são críticos para o sucesso do sistema.



O QUE SÃO CARACTERÍSTICAS ARQUITETURAIS?

- ❑ Definição: São atributos de qualidade do software que não fazem parte das funcionalidades de negócio, mas definem como o sistema se comporta ou quão bem ele realiza suas funções. Elas englobam requisitos de desempenho, segurança, manutenibilidade, entre outros.
- ❑ Terminologia: Também conhecidas como requisitos não-funcionais ou atributos de qualidade. Porém, o termo “não-funcional” pode ser enganoso, pois sugere algo de menor importância; na verdade, essas características são vitais para o sucesso do sistema. Muitos arquitetos preferem o termo características arquiteturais exatamente para enfatizar sua relevância.
- ❑ Exemplos Gerais: Desempenho, escalabilidade, disponibilidade, segurança, usabilidade, entre outras. Por exemplo, um sistema web pode exigir alta disponibilidade (99.9% do tempo no ar) e baixa latência nas respostas. Essas são características arquiteturais que precisam ser atendidas pelo design.



CRITÉRIOS PARA SER UMA CARACTERÍSTICA ARQUITETURAL

- ❑ Fora do domínio de negócio: A preocupação não está ligada à lógica de negócio em si, e sim a considerações transversais de design ou operação. Em outras palavras, não descreve o que o sistema faz para o usuário, mas condições de sucesso sobre como o sistema deve ser construído ou funcionar. Ex.: “tempo de resposta menor que 1s” ou “uso de memória abaixo de 70%” são metas de qualidade, não funcionalidades de negócio.
- ❑ Influencia a estrutura do sistema: O atributo requer decisões arquiteturais específicas, afetando a organização ou componentes do software. Se uma característica é importante o bastante, o arquiteto pode precisar ajustar a estrutura do sistema para atendê-la. Ex.: Requisito de segurança elevado pode levar à criação de um módulo dedicado (ou serviço) para criptografar dados ou processar pagamentos de forma isolada. Ou seja, a característica direciona o design de módulos, camadas, serviços, etc.



CRITÉRIOS PARA SER UMA CARACTERÍSTICA ARQUITETURAL

- ❑ Crítica para o sucesso da aplicação: Atingir determinado nível desse atributo é fundamental para o sistema cumprir sua missão. Cada característica arquitetural adicionada aumenta a complexidade, então focamos apenas nas que realmente importam para o sucesso do produto. Ex.: Em um sistema de negociação financeira de alta frequência, baixa latência é crítica (sem ela, o sistema falha no propósito); já outras qualidades menos críticas podem ser deliberadamente mantidas em um nível “bom o suficiente”.



CARACTERÍSTICAS IMPLÍCITAS VS. EXPLÍCITAS

- ❑ Explícitas: São aquelas declaradas claramente em documentos de requisitos, contratos ou pelos stakeholders. Ou seja, a equipe de negócio ou cliente pede explicitamente certas qualidades. Exemplo: “O sistema deve suportar até 5000 usuários simultâneos” (isso aponta para a característica escalabilidade) ou “Aplicar as normas da LGPD” (característica de privacidade/compliance). Essas virão listadas como requisitos de desempenho, segurança, conformidade etc.
- ❑ Implícitas: São qualidades não mencionadas diretamente, mas que são esperadas naturalmente ou necessárias devido ao contexto do sistema. Muitas vezes, ninguém escreve essas exigências, mas o arquiteto deve inferi-las. Exemplos: Nenhum documento de requisitos de um banco talvez diga “o sistema deve ser seguro contra invasão”, mas obviamente espera-se segurança robusta. Ou um sistema de e-commerce não funciona se não tiver alta disponibilidade: mesmo que não esteja escrita, é implícita dada a natureza 24/7 do negócio.



CARACTERÍSTICAS IMPLÍCITAS VS. EXPLÍCITAS

- ❑ Descoberta proativa: Cabe ao arquiteto levantar essas características implícitas. Isso envolve experiência prévia, conhecimento do domínio e muita comunicação com stakeholders. Entrevistar usuários e patrocinadores ajuda a revelar expectativas não ditas (“precisamos de confiança de que não vai cair o sistema”), e o arquiteto traduz isso em características arquiteturais concretas. Assim, garantimos que nenhum requisito crítico fique esquecido apenas por não ter sido explicitado formalmente.



CARACTERÍSTICAS OPERACIONAIS

- ❑ Desempenho (Performance): Capacidade do sistema de responder rapidamente e processar a carga de trabalho esperada. Inclui tempos de resposta, throughput (transações por segundo) e uso eficiente de recursos computacionais. Exemplo: A aplicação deve processar requisições em < 200ms e suportar picos de 1000 req/segundo sem degradação perceptível. Isso exige otimizações de código, consultas eficientes e talvez testes de estresse para validar.
- ❑ Disponibilidade: Quanto tempo o sistema fica disponível para uso (uptime). Sistemas críticos podem exigir 99,9% ou 99,99% de uptime. Exemplo: Se for 24/7, precisamos planejar redundância e recuperação rápida em caso de falhas. Técnicas como deploy em clusters, failover automático e monitoramento ajudam a atingir alta disponibilidade.
- ❑ Escalabilidade: Habilidade de crescer em capacidade conforme aumenta a demanda (usuários, transações) sem perda de desempenho. Pode ser horizontal (adicionar mais servidores/nós) ou vertical (melhorar hardware de um servidor). Exemplo: Uma aplicação cloud native com auto-scaling pode instanciar novas máquinas conforme o tráfego sobe, garantindo escalabilidade.



CARACTERÍSTICAS OPERACIONAIS

- ❑ Confiabilidade: Frequência e impacto de falhas no sistema. Um sistema confiável opera de forma consistente, sem erros frequentes, e seguro contra perda de dados ou corrupção. Inclui aspectos de segurança de vida e financeiros: se o sistema falhar, causa danos materiais ou riscos humanos?. Exemplo: Numa aplicação médica, confiabilidade é fundamental : ela deve funcionar corretamente sob condições adversas e lidar bem com falhas de componentes.
- ❑ Continuidade de Negócio: Capacidade de recuperação diante de desastres ou interrupções severas, minimizando downtime. Exemplo: Ter um plano de Disaster Recovery (DR): backups regulares, replicação de dados em outra região, infraestrutura redundante. Isso garante que, mesmo se houver uma pane geral ou desastre natural, o sistema possa ser restaurado (ainda que com performance reduzida) em, digamos, menos de 1 hora.
- ❑ Robustez: Habilidade de lidar com condições de erro e situações inesperadas em tempo de execução. Um sistema robusto continua operando (talvez em modo degradado) mesmo se partes falharem. Exemplo: Se um serviço externo ficar fora do ar, o sistema devolve uma resposta cacheada ao invés de travar; se ocorrer um pico anormal de uso, o sistema faz throttling (auto-limitação) em vez de colapsar completamente.



CARACTERÍSTICAS ESTRUTURAIS

- ❑ Modularidade & Manutenibilidade: Capacidade de evoluir e dar manutenção no software de forma fácil e segura. Arquiteturas modulares (baixa dependência entre componentes, alta coesão) permitem atualizar uma parte sem quebrar o resto. Exemplo: Seguir princípios de Design (SOLID, separação de camadas) para que adicionar uma nova funcionalidade ou corrigir um bug seja rápido e de baixo risco. Manutenibilidade elevada implica código claro, bem estruturado e testável.
- ❑ Extensibilidade: Facilidade de adicionar novas funcionalidades ou variar comportamentos existentes sem mudanças drásticas na base de código. Isso geralmente significa ter pontos de extensão bem definidos (p.ex., plugins, módulos desacoplados por interfaces). Exemplo: Uma plataforma de e-commerce permite novos métodos de pagamento plugáveis, em vez de precisar reescrever partes do sistema para suportar o método novo.
- ❑ Reutilização de Componentes: Grau em que partes do sistema podem ser reutilizadas em outros contextos ou produtos. Promover reutilização reduz esforço duplicado. Exemplo: Criar um serviço de autenticação central que pode ser usado por vários sistemas da empresa, ao invés de cada um implementar o seu. Ou aproveitar bibliotecas/módulos comuns (um módulo de envio de e-mail reaproveitado em diferentes aplicativos).



CARACTERÍSTICAS ESTRUTURAIS

- ❑ Portabilidade: Habilidade do sistema de operar em diferentes plataformas tecnológicas com pouca ou nenhuma modificação. Pode ser portar entre sistemas operacionais, bancos de dados, ou ambientes de nuvem diferentes. Exemplo: Uma aplicação Java EE que rode tanto em servidor WildFly quanto em WebLogic, ou um aplicativo desenhado para ser multi-cloud (poder rodar na AWS ou Azure). Alto nível de portabilidade normalmente exige abstração de detalhes de plataforma.
- ❑ Configurabilidade: Facilidade de ajustar o comportamento do software através de configurações externas, sem necessidade de alterar o código-fonte. Exemplo: Uso de arquivos de propriedade, XML/YAML ou variáveis de ambiente para configurar parâmetros como conexões de banco, limites de uso, flags de funcionalidades. Um sistema configurável pode atender diferentes clientes ou cenários apenas mudando configurações, em vez de gerar um novo build.
- ❑ Suportabilidade (Supportability): Nível de facilidade para suportar, monitorar e depurar o sistema em produção. Engloba ter logging adequado, métricas, mecanismo de rastreamento de erros (stack traces claros) e documentação de operação. Exemplo: Inserir logs estruturados e monitoramento desde o desenvolvimento, para que a equipe de suporte consiga identificar problemas em produção rapidamente. Também abrange facilidade de instalar atualizações (upgradeability) e ferramentas de diagnóstico embutidas.



CARACTERÍSTICAS TRANSVERSAIS (CROSS-CUTTING)

- ❑ **Segurança:** Conjunto abrangente de mecanismos para proteger o sistema e os dados contra acessos não autorizados e ataques. Envolve Autenticação (verificar identidades dos usuários) e Autorização (assegurar que cada usuário acesse apenas o que tem permissão) como partes essenciais. Também inclui confidencialidade e integridade dos dados: criptografia de dados sensíveis (em trânsito e em repouso) e medidas contra invasões (firewalls, sanitização de entradas, etc.). Exemplo: Uma aplicação corporativa pode usar OAuth2/ OpenID Connect para autenticação, controlar acessos por perfis de usuário e criptografar todas as comunicações (HTTPS/TLS) e dados em banco.
- ❑ **Privacidade:** Proteção dos dados pessoais e sensíveis, garantindo que apenas quem deve tenha acesso às informações. Relaciona-se a segurança, mas foca em restrição e anonimização de informações dentro do próprio sistema também. Exemplo: Em uma base de dados, dados de clientes podem ser cifrados de forma que nem administradores do banco (DBAs) consigam ler informações confidenciais (data-at-rest encryption). Logs e relatórios devem ocultar ou mascarar informações pessoais identificáveis (PII) para preservar a privacidade.



CARACTERÍSTICAS TRANSVERSAIS (CROSS-CUTTING)

- ❑ Compliance Legal: Adequação a leis e regulamentações externas que impactam o sistema. Pode incluir normas de proteção de dados (LGPD, GDPR), requisitos de auditoria financeira (SOX - Sarbanes-Oxley), normas específicas de saúde (HIPAA) etc. Exemplo: Um sistema sujeito à GDPR deve prever recursos para atender direitos do usuário, como deletar dados pessoais mediante solicitação, guardar consentimentos, além de registrar acessos (auditoria) para provar conformidade. Do ponto de vista arquitetural, compliance pode exigir módulos de auditoria e arquivamento de dados e controles de acesso muito bem definidos.
- ❑ Acessibilidade: Qualidade de um sistema ser utilizável por pessoas com deficiência ou necessidades especiais. Isso atravessa a camada de interface e lógica: inclui suporte a leitores de tela (por meio de HTML semântico, por exemplo), contraste de cores adequado para daltônicos, navegação via teclado para quem não pode usar mouse, legendas em conteúdos audiovisuais etc. Exemplo: Em uma aplicação web, seguir as diretrizes WCAG (Web Content Accessibility Guidelines), fornecer textos alternativos para imagens, permitir aumento de fonte, garantir que todas as funcionalidades estejam disponíveis pelo teclado.



CARACTERÍSTICAS TRANSVERSAIS (CROSS-CUTTING)

- ❑ Usabilidade: Medida de quão fácil e intuitivo é para usuários atingirem seus objetivos no sistema. Abrange aspectos como interface amigável, consistência de fluxos, feedback claro ao usuário e baixa necessidade de treinamento. Do ponto de vista arquitetural, pode afetar decisões como adoção de certos frameworks UI ou padrões de design. Exemplo: Um software interno pode ter como requisito de usabilidade que novos usuários aprendam a operá-lo em menos de 1 hora (o que implica em design simples, documentação in-app, etc.). Obs: Usabilidade às vezes não é vista como “arquitetural”, mas quando tratada seriamente, exige esforço de design de interação, e pode impactar escolha de tecnologias (por isso listamos aqui entre características transversais).
- ❑ Arquivamento e Retenção: Necessidade de armazenar, arquivar ou descartar dados antigos de acordo com regras de negócio ou legais. É transversal pois afeta banco de dados, aplicações e processos. Exemplo: “Logs devem ser mantidos por 6 meses e então apagados”; ou “Clientes inativos por mais de 5 anos devem ter contas arquivadas em um banco separado”. Arquiteturalmente, isso implica talvez ter um sistema de arquivamento (jobs que movem dados velhos para outro local) ou estratégias de particionamento de dados por tempo, etc.



TRADE-OFFS ENTRE CARACTERÍSTICAS

- ❑ Interdependência: As características arquiteturais frequentemente competem entre si. Melhorar uma pode prejudicar outra. Exemplo clássico: aumentar segurança adicionando criptografia forte e mais verificações certamente impacta a performance (mais processamento e passos extras). Outro: exigir alta consistência de dados (transações rígidas) pode reduzir a escalabilidade em sistemas distribuídos.
- ❑ Complexidade Acumulada: Cada característica extra suportada torna o design mais complexo. Portanto, não é viável tentar maximizar todas as qualidades ao mesmo tempo. Adicionar muitas “-ilities” resulta em arquitetura genérica, pesada, difícil de implementar e manter. Arquitetos precisam reconhecer limites práticos e evitar atender requisitos de qualidade desnecessários, sob risco de o sistema ficar inviável.



TRADE-OFFS ENTRE CARACTERÍSTICAS

- ❑ **Balanço e Compromisso:** O trabalho do arquiteto é encontrar um equilíbrio apropriado conforme as prioridades do negócio. Exemplo: se desempenho em tempo real é prioridade #1 e segurança #2, talvez se aceite usar algoritmos de criptografia mais leves (comprometendo um pouco a segurança, mas melhorando performance). Ou, se portabilidade não é importante para o produto, pode-se decidir por tecnologias específicas altamente performáticas (sacrifica portabilidade em prol de desempenho). Cada decisão arquitetural requer avaliar esses trade-offs e justificar a escolha em função dos objetivos principais.



FOCO NAS PRIORIDADES

- ❑ Menos pior, não perfeita: “Never shoot for the best architecture, but rather the least worst architecture”. Nunca mire na arquitetura “perfeita” em tudo, mas na que minimize arrependimentos. A ideia é ser pragmático: atenda muito bem aos poucos requisitos arquiteturais que mais importam e não deixe que o resto atrapalhe. Em outras palavras, otimize para as prioridades e seja “bom o suficiente” nos demais aspectos.
- ❑ Escolha de Poucas Características-Chave: Em vez de tentar abraçar dez qualidades, eleja um conjunto enxuto (geralmente 3-5) de características arquiteturais essenciais que o sistema deve exibir em alto nível. Foque nelas durante o design. Suportar o mínimo necessário de características resulta numa solução mais simples e coerente. Dica: Pode-se pedir aos stakeholders para listar as 3 qualidades mais importantes; isso ajuda a guiar o arquiteto e gera consenso sobre o foco.



FOCO NAS PRIORIDADES

- ❑ Adaptabilidade & Iteração: Projetar a arquitetura de forma que mudanças futuras sejam possíveis (arquitetura evolutiva) diminui a necessidade de “prever tudo agora”. Se novas características se tornarem críticas depois, poderemos incorporá-las. Ou seja, construir com princípios ágeis: a arquitetura deve suportar iterações e melhorias contínuas, em vez de tentar ser perfeita desde o início. Com isso, reduz-se a ansiedade de acertar 100% de primeira. Podemos evoluir conforme aprendemos mais, evitando exageros prematuros.



IDENTIFICANDO CARACTERÍSTICAS ARQUITETURAIS

- ❑ **Extraíndo do Domínio:** Comece entendendo profundamente as prioridades do negócio e do domínio do problema. Cada objetivo ou preocupação de negócio pode ser traduzido em “-ilidades” técnicas. Exemplo: Se o negócio diz “precisamos atingir o mercado rapidamente com novas features”, o arquiteto traduz para uma necessidade de manutenibilidade/deploy contínuo (entrega rápida -> arquitetura flexível e fácil de modificar). Se o contexto é “usuários espalhados globalmente”, isso sugere requisitos de escalabilidade e baixa latência. Essa ponte entre linguagem de negócio e termos técnicos é o primeiro passo.
- ❑ **Analisando Requisitos Explícitos:** Verifique documentos de requisitos, contratos de SLA ou histórias de usuário em busca de critérios quantificáveis ou restrições que impliquem características arquiteturais. Números e condições específicas são pistas: “até 2 segundos por resposta” (desempenho), “suportar crescimento de 10x usuários em 1 ano” (escalabilidade), “conformidade PCI para dados de cartão” (segurança). Liste esses requisitos não-funcionais explícitos e garanta que serão tratados no design.



IDENTIFICANDO CARACTERÍSTICAS ARQUITETURAIS

- ❑ **Conhecimento Implícito & Padrões do Setor:** Aplique a experiência e boas práticas do domínio para identificar características não solicitadas mas essenciais. Baseie-se em problemas semelhantes: sistemas financeiros exigem integridade transacional e baixa latência; aplicações médicas precisam de alta confiabilidade; serviços públicos web demandam acessibilidade ampla, etc. Mesmo que o cliente não peça, o arquiteto incorpora esses requisitos implícitos para evitar riscos futuros. Dica: Revisite listas de atributos comuns (como vimos: disponibilidade, segurança, etc.) e pergunte-se se cada um é relevante ao projeto em questão.
- ❑ **Envolvendo os Stakeholders:** Faça workshops ou entrevistas com as partes interessadas (clientes, usuários-chave, patrocinadores) para descobrir o que é realmente importante para eles. Muitas vezes eles expressam em termos de negócio (ex.: “precisamos ser os primeiros no mercado” = rapidez de entrega; “o usuário tem que adorar usar” = usabilidade). Cabe ao arquiteto traduzir essas metas em características arquiteturais e confirmar com eles. Além disso, essa conversa serve para priorizar: peça para ordenarem o que mais importa (como dito, top 3). Isso alinha expectativas e dá um direcionamento claro. Documente as características resultantes dessa análise para guiar as decisões de arquitetura.



O CASO DO NAVIO VASA (1628)

- ❑ **A ambição Excessiva:** O navio de guerra Vasa, encomendado pelo Rei da Suécia em 1628, foi projetado para ser o mais formidável já construído. O rei insistiu em todas as características “máximas”: duas fileiras de canhões (em vez de uma, comum na época), canhões muito maiores que o padrão, e ao mesmo tempo servir tanto como navio de guerra quanto de transporte de tropas. Em termos de software, seria como exigir simultaneamente desempenho máximo, segurança máxima, features extras... tudo no mesmo projeto.
- ❑ **Complexidade Fatal:** O resultado foi um navio extremamente pesado e instável. Em sua viagem inaugural, diante de convidados, uma rajada de vento bastou para desequilibrá-lo. O Vasa virou e afundou no porto antes mesmo de iniciar sua missão. Toda a grandiosidade planejada foi por água abaixo (literalmente) devido ao desequilíbrio de prioridades no projeto.
- ❑ **Lição Arquitetural:** Assim como no Vasa, tentar suportar características demais em grau máximo pode arruinar um sistema. Projetos de software que acumulam todas as exigências imagináveis tendem a falhar, seja por atraso, custo excessivo ou complexidade impossível de gerir. A história do Vasa nos lembra da importância de escolher as batalhas certas. Priorizar os requisitos arquiteturais realmente importantes e projetar dentro das limitações práticas. Em vez de um “navio que faz tudo e afunda”, construímos um que faz bem aquilo a que se propõe.



CONCLUSÃO

- ❑ Importância Vital: Características arquiteturais não são “detalhes opcionais”, mas sim fatores essenciais para o sucesso do sistema. São elas que garantem que o produto, além de fazer o que precisa, o faça bem: com desempenho adequado, confiabilidade, segurança, etc. Em projetos de software profissionais, espera-se que o arquiteto trate essas características no mesmo nível de importância dos requisitos funcionais.
- ❑ Drivers do Design: Desde as primeiras definições da arquitetura, devemos ter em mente os atributos de qualidade prioritários, pois eles orientam decisões estruturais. A escolha de arquitetura em camadas, microsserviços, uso de cache, redundância, frameworks, tudo isso deve ser guiado pelas características arquiteturais desejadas. Elas funcionam como critérios de projeto: diferentes arquiteturas serão avaliadas pelo quanto atendem às “ilities” requeridas.



CONCLUSÃO

- ❑ Identificação e Consenso: A seleção das características arquiteturais deve ser um processo colaborativo e consciente. Envolve analisar requisitos explícitos, captar necessidades implícitas e, principalmente, conversar com stakeholders para alinhar expectativas. O resultado deve ser um consenso sobre quais qualidades o sistema deve ter e em que grau. Assim evitamos tanto esquecer algo crítico quanto tentar abraçar demais. Encontramos o foco certo.
- ❑ Projeto Equilibrado e Evolutivo: Uma vez definidas as características-chave, o arquiteto busca um equilíbrio: satisfazer ao máximo essas prioridades sem complicar desnecessariamente outras áreas. A arquitetura resultante é propositalmente simples onde pode ser, e complexa apenas onde precisa ser (para atender as qualidades escolhidas). Além disso, adota-se uma postura de evolução contínua: monitora-se as características durante o desenvolvimento (e até após, em produção) e ajusta-se o design conforme necessário, através de refatorações arquiteturais ou uso de fitness functions (métricas automatizadas de qualidade, se aplicável). Dessa forma, a arquitetura permanece saudável e alinhada aos objetivos de negócio ao longo do tempo.



ATÉ A PRÓXIMA AULA